Objective 3

Elda

2026-10-20

Loading packages

```
library(terra)
```

```
## terra 1.9.11
```

```
library(sf)
```

```
## Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:terra':
##
##     intersect, union
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(mgcv)
```

```
## Loading required package: nlme
```

```
##
## Attaching package: 'nlme'
```

```
## The following object is masked from 'package:dplyr':
##
##     collapse
```

```
## This is mgcv 1.9-4. For overview type '?mgcv'.
```

```r
library(spdep)
```

```
## Loading required package: spData

## To access larger datasets in this package, install the spDataLarge
## package with: 'install.packages('spDataLarge',
## repos='https://nowosad.github.io/drat/', type='source')'
```

```r
library(PRROC)
```

```
## Loading required package: rlang
```

```r
# Folder containing the three crime-stage GeoPackages
OBJECTIVE3_DIR <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangoli

INPUT_DIR <- dirname(OBJECTIVE3_DIR)

# Crime-stage files
stage_files <- c(
  Stage_I   = file.path(OBJECTIVE3_DIR, "Crime_stage_I.gpkg"),
  Stage_II  = file.path(OBJECTIVE3_DIR, "Crime_stage_II.gpkg"),
  Stage_III = file.path(OBJECTIVE3_DIR, "Crime_stage_III.gpkg")
)

# The exact nine covariates used in the previous analysis
covariate_names <- c(
  "B_distance_to_Forest_Concessions",
  "B_distance_to_InternationalGabonBorders",
  "B_distance_to_MajorGabonCities",
  "B_distance_to_majorRoads",
  "B_distance_to_ProtectedAreas",
  "B_distance_to_waterways",
  "B_LAI_2024",
  "B_Population_2020",
  "B_SRTM_2000"
)

# Spatial cross-validation settings
N_FOLDS <- 5
BLOCK_SIZE_M <- 80000 #[m]
SPATIAL_K <- 20 # Maximum allowed complexity (basis dimension) of the 2-D spatial smooth s(x, y)

stopifnot(all(file.exists(stage_files)))

# Knitting runs in a fresh R session, so load the stage data here.
crime_points <- lapply(stage_files, function(f) st_read(f, quiet = TRUE))

# Use the first stage's CRS consistently for the initial vector maps.
if (is.na(st_crs(crime_points[[1]]))) {
  stop("The Stage I point data have no CRS. Define their CRS before proceeding.")
}
crime_points <- lapply(crime_points, function(x) {
```

```
  if (is.na(st_crs(x))) stop("A crime-stage dataset has no CRS.")
  st_transform(x, st_crs(crime_points[[1]]))
})
```

1. Loading and visualizing the 3 crime stages and the 9 covariates

```
# Gabon national boundary
gabon_boundary_file <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pa

# Major roads
roads_file <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_

# National parks / protected areas
parks_file <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_

gabon_boundary <- st_read(gabon_boundary_file, quiet = TRUE)
major_roads <- st_read(roads_file, quiet = TRUE)
national_parks <- st_read(parks_file, quiet = TRUE)

# Ensure all layers use the point-data CRS
target_crs <- st_crs(crime_points[[1]])

gabon_boundary <- st_transform(gabon_boundary, target_crs)
major_roads <- st_transform(major_roads, target_crs)
national_parks <- st_transform(national_parks, target_crs)

# Print point counts
sapply(crime_points, nrow)
```

```
##   Stage_I  Stage_II Stage_III
##       702       763       696
```

```
# Map the three crime stages over the Gabon boundary, national parks, and roads
par(mfrow = c(1, 3), mar = c(2, 2, 3, 1))

stage_cols <- c(
  Stage_I = "#1b9e77",
  Stage_II = "#d95f02",
  Stage_III = "#7570b3"
)

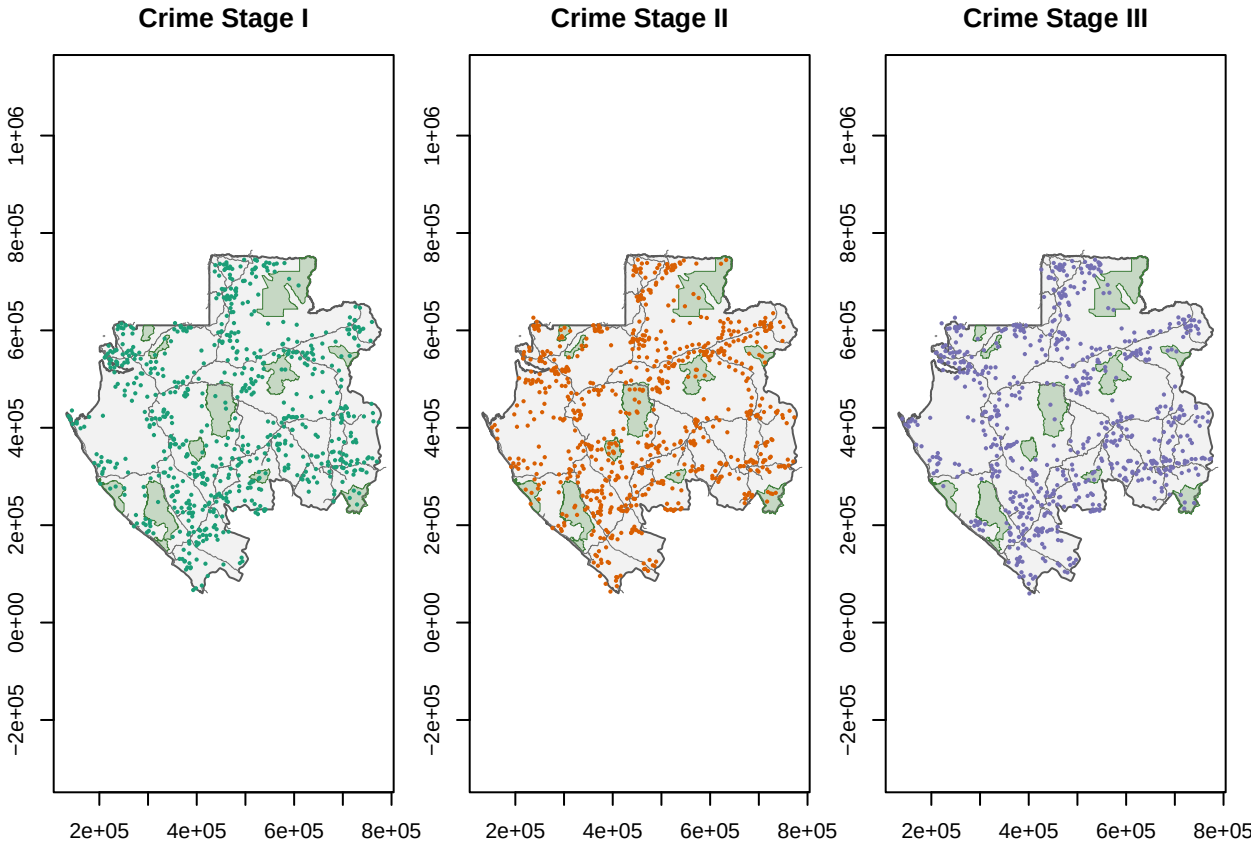
for (stage_name in names(crime_points)) {

  # Gabon outline
  plot(
    st_geometry(gabon_boundary),
    col = "grey95",
    border = "grey35",
    main = paste("Crime", gsub("_", " ", stage_name)),
    axes = TRUE
  )
```

```
  # National parks / protected areas
  plot(
    st_geometry(national_parks),
    col = adjustcolor("#74a96f", alpha.f = 0.35),
    border = "#3e7d3a",
    lwd = 0.5,
    add = TRUE
  )

  # Major roads
  plot(
    st_geometry(major_roads),
    col = "grey45",
    lwd = 0.35,
    add = TRUE
  )

  # Participatory-mapped points
  plot(
    st_geometry(crime_points[[stage_name]]),
    col = stage_cols[stage_name],
    pch = 16,
    cex = 0.45,
    add = TRUE
  )
}
```




```
par(mfrow = c(1, 1))
```

2. Loading and visualizing the 9 covariates

```
# Search both Input and Objective_3 recursively for GeoTIFF files.
search_folders <- unique(c(INPUT_DIR, OBJECTIVE3_DIR))

all_raster_files <- unlist(
  lapply(search_folders, function(folder) {
    list.files(
      folder,
      pattern = "\\.(tif|tiff)$",
      full.names = TRUE,
      recursive = TRUE,
      ignore.case = TRUE
    )
  })
)

all_raster_files <- unique(all_raster_files)

if (length(all_raster_files) == 0) {
  stop(
    "No .tif or .tiff files were found under:\n",
    paste(search_folders, collapse = "\n")
```

```
  )
}

# Remove file extension and match each expected covariate name.
raster_base_names <- tools::file_path_sans_ext(basename(all_raster_files))

cov_files <- sapply(covariate_names, function(covariate) {

  matching_file <- all_raster_files[
    tolower(raster_base_names) == tolower(covariate)
  ]

  if (length(matching_file) == 0) {
    return(NA_character_)
  }

  matching_file[1]
})

names(cov_files) <- covariate_names

# Stop and tell you exactly which covariate file was not located.
if (any(is.na(cov_files))) {
  missing_covariates <- names(cov_files)[is.na(cov_files)]

  cat("These raster files were found:\n")
  print(all_raster_files)

  stop(
    "\nCould not find these expected covariate rasters:\n",
    paste(missing_covariates, collapse = "\n"),
    "\n\nCheck whether their filenames differ from the names in covariate_names."
  )
}

# Print the exact files used.
cat("The following nine covariate rasters will be used:\n")
```

```
## The following nine covariate rasters will be used:
```

```
print(cov_files)
```

```
##
##        "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_Gabo
##
## "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_Gabon/Input
##
##          "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_Ga
##
##                "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangol
##
##            "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_
```

```
##
##                 "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pango
##
##                              "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/V
##
##                       "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures
##
##                             "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ve
```

```r
# Verify each input grid before stacking.
cov_layers <- lapply(unname(cov_files), terra::rast)
alignment_ok <- vapply(cov_layers, function(layer) {
  terra::compareGeom(cov_layers[[1]], layer, stopOnError = FALSE)
}, logical(1))
if (!all(alignment_ok)) {
  stop("Covariate grids are not aligned: ",
       paste(covariate_names[!alignment_ok], collapse = ", "))
}
cat("All nine covariate grids are aligned.\n")
```

```
## All nine covariate grids are aligned.
```

```r
# Load rasters in the intended covariate order.
covs <- rast(cov_layers)
names(covs) <- covariate_names

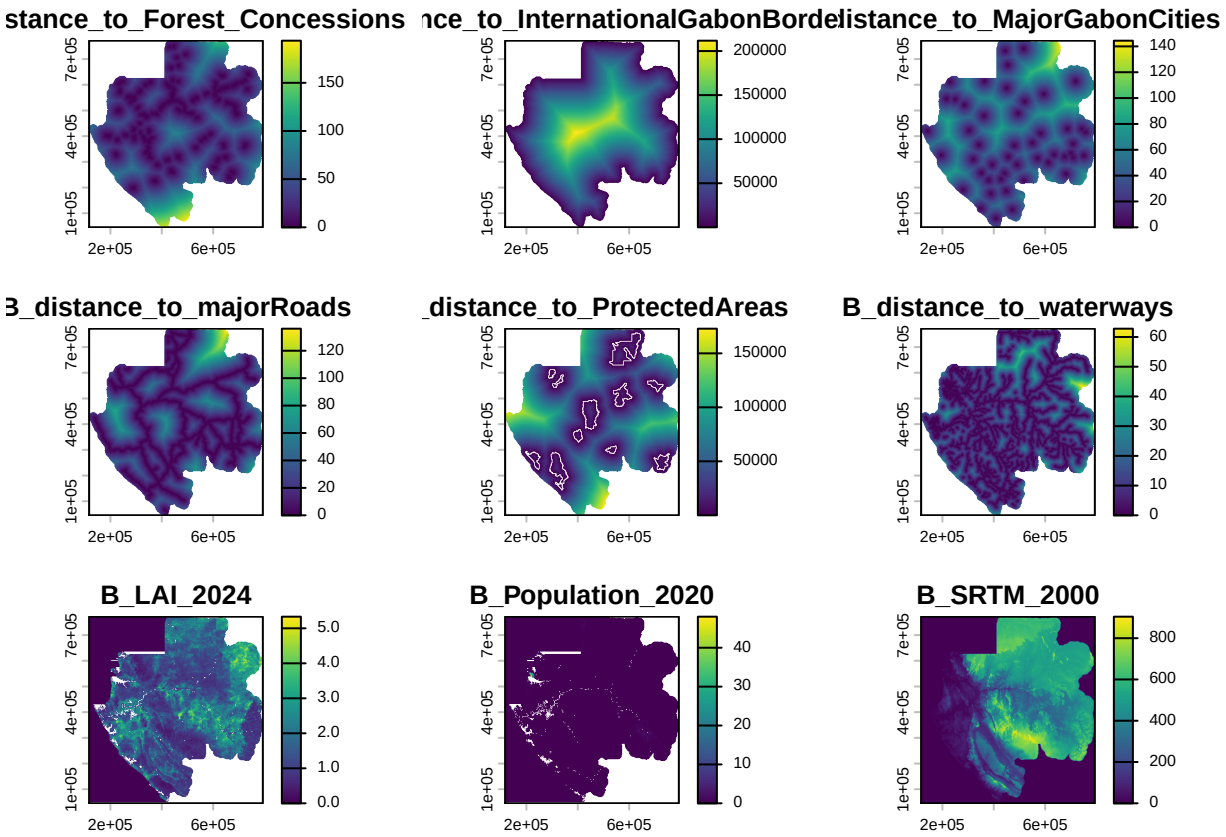
# Reproject points to the covariate grid before rasterization.
crime_points <- lapply(crime_points, function(x) st_transform(x, crs(covs)))

print(covs)
```

```
## class       : SpatRaster
## size        : 363, 338, 9  (nrow, ncol, nlyr)
## resolution  : 2001.067, 2001.262  (x, y)
## extent      : 117119.5, 793480.2, 45184.08, 771642.2  (xmin, xmax, ymin, ymax)
## coord. ref. : WGS 84 / Gabon TM (EPSG:5223)
## sources     : B_distance_to_Forest_Concessions.tif
##               B_distance_to_InternationalGabonBorders.tif
##               B_distance_to_MajorGabonCities.tif
##               ... and 6 more sources
## names       : B_dis~sions, B_dis~rders, B_dis~ities, B_dis~Roads, B_dis~Areas, B_dis~rways, ...
## min values  :      0.0000,          ? ,      0.0000,      0.0000,          ? ,     0.00000, ...
## max values  :    193.7628,          ? ,    144.4238,    136.0873,          ? ,    62.80292, ...
```

```r
# Map all nine covariates.
par(mfrow = c(3, 3), mar = c(2, 2, 3, 4))

for (j in seq_len(nlyr(covs))) {
  plot(covs[[j]], main = names(covs)[j])
}
```




```
par(mfrow = c(1, 1))
```

3. Check CRS, grid alignment, extent, resolution, and covariate correlation

```
# 2. Spearman correlation: calculate without printing the full matrix
cov_values <- as.data.frame(covs, na.rm = TRUE)

cor_matrix <- cor(
  cov_values,
  method = "spearman",
  use = "complete.obs"
)

# 3. Keep each covariate pair once; remove diagonal self-correlations
cor_pairs <- as.data.frame(as.table(cor_matrix), stringsAsFactors = FALSE) |>
  dplyr::rename(
    variable_1 = Var1,
    variable_2 = Var2,
    spearman_rho = Freq
  ) |>
  dplyr::mutate(
    abs_rho = abs(spearman_rho),
    keep_pair = match(variable_1, colnames(cor_matrix)) <
      match(variable_2, colnames(cor_matrix))
  ) |>
```

```
  dplyr::filter(keep_pair) |>
  dplyr::select(-keep_pair)

# Most negative, most positive, and strongest absolute correlation
min_pair <- cor_pairs |>
  dplyr::slice_min(spearman_rho, n = 1, with_ties = FALSE)

max_pair <- cor_pairs |>
  dplyr::slice_max(spearman_rho, n = 1, with_ties = FALSE)

strongest_pair <- cor_pairs |>
  dplyr::slice_max(abs_rho, n = 1, with_ties = FALSE)

cat("\n================ SPEARMAN CORRELATION SUMMARY ================\n")
```

```
##
## ================ SPEARMAN CORRELATION SUMMARY ================
```

```
cat(
  "\nMinimum correlation:\n  rho = ", round(min_pair$spearman_rho, 3),
  "\n  Variables: ", min_pair$variable_1,
  " and ", min_pair$variable_2, "\n",
  sep = ""
)
```

```
##
## Minimum correlation:
##   rho = -0.33
##   Variables: B_distance_to_majorRoads and B_Population_2020
```

```
cat(
  "\nMaximum correlation:\n  rho = ", round(max_pair$spearman_rho, 3),
  "\n  Variables: ", max_pair$variable_1,
  " and ", max_pair$variable_2, "\n",
  sep = ""
)
```

```
##
## Maximum correlation:
##   rho = 0.608
##   Variables: B_distance_to_MajorGabonCities and B_distance_to_majorRoads
```

```
cat(
  "\nLargest absolute correlation used for collinearity screening:\n  |rho| = ",
  round(strongest_pair$abs_rho, 3),
  "\n  Variables: ", strongest_pair$variable_1,
  " and ", strongest_pair$variable_2, "\n",
  sep = ""
)
```

```
##
```

```
## Largest absolute correlation used for collinearity screening:
##   |rho| = 0.608
##   Variables: B_distance_to_MajorGabonCities and B_distance_to_majorRoads
```

```
if (strongest_pair$abs_rho < 0.70) {
  cat("\nNo pair exceeded |rho| = 0.70; all covariates are retained.\n")
} else {
  cat("\nAt least one pair reached |rho| = 0.70; review correlated covariates.\n")
}
```

```
##
## No pair exceeded |rho| = 0.70; all covariates are retained.
```

4. Convert points to binary occurrence maps on the covariate grid

```
template_raster <- covs[[1]]

# Keep only cells where every covariate has a non-missing value.
valid_mask <- app(covs, fun = function(x) as.integer(all(!is.na(x))))

stage_counts <- list()
stage_occurrence <- list()

for (stage_name in names(crime_points)) {

  pts <- crime_points[[stage_name]]
  pts$one_point <- 1

  # Count points in each covariate-grid cell.
  count_raster <- rasterize(
    vect(pts),
    template_raster,
    field = "one_point",
    fun = "sum",
    background = 0
  )

  # Convert counts to binary occurrence: 0 = no point, 1 = >= 1 point.
  occurrence_raster <- ifel(count_raster > 0, 1, 0)

  # Remove cells lacking any covariate value.
  count_raster <- mask(count_raster, valid_mask, maskvalues = 0)
  occurrence_raster <- mask(occurrence_raster, valid_mask, maskvalues = 0)

  names(count_raster) <- paste0(stage_name, "_count")
  names(occurrence_raster) <- paste0(stage_name, "_occurrence")

  stage_counts[[stage_name]] <- count_raster
  stage_occurrence[[stage_name]] <- occurrence_raster

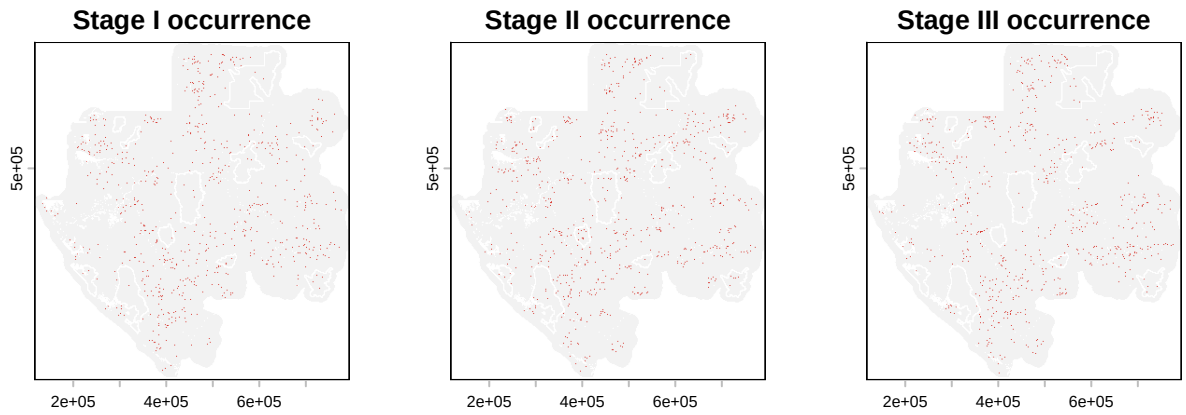
  # Console summary
  n_valid_cells <- global(!is.na(occurrence_raster), "sum", na.rm = TRUE)[1, 1]
  n_positive_cells <- global(occurrence_raster, "sum", na.rm = TRUE)[1, 1]
```

```r
  cat("\n", stage_name, "\n", sep = "")
  cat("Total participatory-mapped points:", nrow(pts), "\n")
  cat("Positive covariate-grid cells:", n_positive_cells, "\n")
  cat("Valid covariate-grid cells:", n_valid_cells, "\n")
  cat("Percentage of cells containing >=1 mapped point:",
      round(100 * n_positive_cells / n_valid_cells, 3), "%\n")
}
```

```
##
## Stage_I
## Total participatory-mapped points: 702
## Positive covariate-grid cells: 662
## Valid covariate-grid cells: 73236
## Percentage of cells containing >=1 mapped point: 0.904 %
##
## Stage_II
## Total participatory-mapped points: 763
## Positive covariate-grid cells: 734
## Valid covariate-grid cells: 73236
## Percentage of cells containing >=1 mapped point: 1.002 %
##
## Stage_III
## Total participatory-mapped points: 696
## Positive covariate-grid cells: 662
## Valid covariate-grid cells: 73236
## Percentage of cells containing >=1 mapped point: 0.904 %
```

```r
# Map the binary occurrence rasters
par(mfrow = c(1, 3), mar = c(2, 2, 3, 2))

for (stage_name in names(stage_occurrence)) {
  plot(
    stage_occurrence[[stage_name]],
    col = c("grey95", "#d73027"),
    breaks = c(-0.5, 0.5, 1.5),
    legend = FALSE,
    main = paste(gsub("_", " ", stage_name), "occurrence")
  )
  legend(
    "bottomleft",
    legend = c("No mapped point", ">=1 mapped point"),
    fill = c("grey95", "#d73027"),
    bty = "n"
  )
}
```




```
par(mfrow = c(1, 1))

# Folder for exported maps
OUTPUT_DIR <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_

# Create the folder if it does not already exist
if (!dir.exists(OUTPUT_DIR)) {
  dir.create(OUTPUT_DIR, recursive = TRUE)
}
```

5. Standardize covariates, create spatial folds, and perform spatial cross-validation

```
# Standardize covariates using all valid Gabon grid cells.
cov_means <- sapply(cov_values, mean, na.rm = TRUE)
cov_sds <- sapply(cov_values, sd, na.rm = TRUE)

covs_z <- covs

for (j in seq_len(nlyr(covs))) {
  covs_z[[j]] <- (covs[[j]] - cov_means[j]) / cov_sds[j]
}

names(covs_z) <- paste0("z_", names(covs))

# Create one modelling table containing coordinates, standardized covariates,
```

```r
# and the three binary occurrence outcomes.
model_rasters <- c(covs_z, rast(stage_occurrence))

model_df <- as.data.frame(
  model_rasters,
  xy = TRUE,
  na.rm = TRUE
)

# Make regular preliminary spatial blocks, then group them into five
# broad, geographically compact cross-validation folds.
model_df$block_x <- floor((model_df$x - min(model_df$x)) / BLOCK_SIZE_M)
model_df$block_y <- floor((model_df$y - min(model_df$y)) / BLOCK_SIZE_M)
model_df$small_block <- interaction(model_df$block_x, model_df$block_y, drop = TRUE)

outcome_names <- names(stage_occurrence)
z_covariate_names <- names(covs_z)

block_summary <- model_df |>
  dplyr::group_by(small_block) |>
  dplyr::summarise(
    x = mean(x),
    y = mean(y),
    dplyr::across(dplyr::all_of(outcome_names), sum),
    .groups = "drop"
  )

# Try several seeds and retain the geographically compact fold arrangement
# with the best distribution of positive cells across folds.
best_solution <- NULL
best_minimum <- -Inf

for (seed in 1:100) {
  set.seed(seed)

  km <- kmeans(
    scale(block_summary[, c("x", "y")]),
    centers = N_FOLDS,
    nstart = 20
  )

  candidate <- block_summary
  candidate$fold <- km$cluster

  positive_by_fold <- candidate |>
    dplyr::group_by(fold) |>
    dplyr::summarise(
      dplyr::across(dplyr::all_of(outcome_names), sum),
      .groups = "drop"
    )

  smallest_positive_count <- min(as.matrix(positive_by_fold[, outcome_names]))
```

```
  if (smallest_positive_count > best_minimum) {
    best_minimum <- smallest_positive_count
    best_solution <- candidate
  }
}

model_df <- model_df |>
  dplyr::left_join(
    best_solution[, c("small_block", "fold")],
    by = "small_block"
  )

cat("Number of preliminary", BLOCK_SIZE_M / 1000, "km spatial blocks:",
    nrow(block_summary), "\n")
```

```
## Number of preliminary 80 km spatial blocks: 65
```

```
cat("Number of broad spatial cross-validation folds:", N_FOLDS, "\n\n")
```

```
## Number of broad spatial cross-validation folds: 5
```

```
fold_prevalence <- model_df |>
  dplyr::group_by(fold) |>
  dplyr::summarise(
    n_cells = dplyr::n(),
    dplyr::across(
      dplyr::all_of(outcome_names),
      list(n_positive = ~ sum(.x), prevalence = ~ mean(.x))
    ),
    .groups = "drop"
  )

print(fold_prevalence, n = Inf)
```

```
## # A tibble: 5 x 8
##    fold n_cells Stage_I_n_positive Stage_I_prevalence Stage_II_n_positive
##   <int>   <int>              <dbl>              <dbl>               <dbl>
## 1     1   13921                164            0.0118                 171
## 2     2   13838                 92            0.00665                120
## 3     3   16024                139            0.00867                139
## 4     4   14212                 91            0.00640                120
## 5     5   15241                176            0.0115                 184
## # i 3 more variables: Stage_II_prevalence <dbl>, Stage_III_n_positive <dbl>,
## #   Stage_III_prevalence <dbl>
```

```
# Formula: occurrence ~ nine standardized covariates + broad 2-D spatial surface
gam_formula <- as.formula(
  paste(
    "OUTCOME ~",
    paste(z_covariate_names, collapse = " + "),
    "+ s(x, y, k =", SPATIAL_K, ", bs = 'tp')"
```

```
  )
)

# Function for PR-AUC
calculate_pr_auc <- function(observed, predicted) {
  PRROC::pr.curve(
    scores.class0 = predicted[observed == 1],
    scores.class1 = predicted[observed == 0],
    curve = FALSE
  )$auc.integral
}

# Fit spatial binomial GAMs using held-out spatial folds.
oof_predictions <- list()
fold_metrics <- list()

for (outcome_name in outcome_names) {

  cat("\nRunning spatial cross-validation for", outcome_name, "\n")

  stage_oof <- list()
  stage_metrics <- list()

  for (held_out_fold in sort(unique(model_df$fold))) {

    cat("  Held-out fold", held_out_fold, "of", N_FOLDS, "\n")

    train_df <- model_df |>
      dplyr::filter(fold != held_out_fold)

    test_df <- model_df |>
      dplyr::filter(fold == held_out_fold)

    fold_formula <- update(
      gam_formula,
      paste(outcome_name, "~ .")
    )

    fold_model <- bam(
      formula = fold_formula,
      family = binomial(link = "logit"),
      data = train_df,
      method = "fREML",
      discrete = TRUE
    )

    test_df$predicted_probability <- as.numeric(
      predict(fold_model, newdata = test_df, type = "response")
    )

    observed <- test_df[[outcome_name]]
    predicted <- test_df$predicted_probability
    prevalence <- mean(observed)
```

```
    stage_metrics[[as.character(held_out_fold)]] <- data.frame(
      stage = outcome_name,
      fold = held_out_fold,
      n_cells = nrow(test_df),
      n_positive = sum(observed),
      prevalence = prevalence,
      PR_AUC = calculate_pr_auc(observed, predicted),
      Brier_score = mean((observed - predicted)^2),
      Brier_prevalence_baseline = mean((observed - prevalence)^2)
    )

    stage_oof[[as.character(held_out_fold)]] <- test_df |>
      dplyr::select(
        x, y, fold,
        observed = dplyr::all_of(outcome_name),
        predicted_probability
      )
  }

  stage_oof <- dplyr::bind_rows(stage_oof)
  stage_oof$stage <- outcome_name

  oof_predictions[[outcome_name]] <- stage_oof
  fold_metrics[[outcome_name]] <- dplyr::bind_rows(stage_metrics)
}
```

```
##
## Running spatial cross-validation for Stage_I
##   Held-out fold 1 of 5
##   Held-out fold 2 of 5
##   Held-out fold 3 of 5
##   Held-out fold 4 of 5
##   Held-out fold 5 of 5
##
## Running spatial cross-validation for Stage_II
##   Held-out fold 1 of 5
##   Held-out fold 2 of 5
##   Held-out fold 3 of 5
##   Held-out fold 4 of 5
##   Held-out fold 5 of 5
##
## Running spatial cross-validation for Stage_III
##   Held-out fold 1 of 5
##   Held-out fold 2 of 5
##   Held-out fold 3 of 5
##   Held-out fold 4 of 5
##   Held-out fold 5 of 5
```

```
oof_predictions <- dplyr::bind_rows(oof_predictions)
fold_metrics <- dplyr::bind_rows(fold_metrics)

cat("\nFold-specific spatial cross-validation metrics:\n")
```

```
##
## Fold-specific spatial cross-validation metrics:
```

```
print(fold_metrics, row.names = FALSE)
```

```
##      stage fold n_cells n_positive  prevalence     PR_AUC Brier_score
##    Stage_I    1   13921        164 0.011780763 0.02715679 0.011603682
##    Stage_I    2   13838         92 0.006648360 0.01753263 0.006575849
##    Stage_I    3   16024        139 0.008674488 0.01761000 0.008672128
##    Stage_I    4   14212         91 0.006403040 0.01847251 0.006360710
##    Stage_I    5   15241        176 0.011547799 0.01902283 0.011418188
##   Stage_II    1   13921        171 0.012283600 0.03285070 0.015343473
##   Stage_II    2   13838        120 0.008671773 0.02348955 0.008556072
##   Stage_II    3   16024        139 0.008674488 0.02687964 0.008522804
##   Stage_II    4   14212        120 0.008443569 0.03296664 0.008284082
##   Stage_II    5   15241        184 0.012072699 0.02722748 0.011876009
##  Stage_III    1   13921        156 0.011206092 0.02032718 0.011052935
##  Stage_III    2   13838         87 0.006287036 0.01962338 0.006203721
##  Stage_III    3   16024        155 0.009672991 0.01609127 0.009748009
##  Stage_III    4   14212         92 0.006473403 0.02145390 0.006383694
##  Stage_III    5   15241        172 0.011285349 0.01751193 0.011153133
##  Brier_prevalence_baseline
##                0.011641977
##                0.006604159
##                0.008599242
##                0.006362041
##                0.011414447
##                0.012132713
##                0.008596574
##                0.008599242
##                0.008372275
##                0.011926949
##                0.011080515
##                0.006247509
##                0.009579424
##                0.006431498
##                0.011157990
```

```
# Overall OOF metrics: all held-out predictions combined across folds.
overall_metrics <- oof_predictions |>
  dplyr::group_by(stage) |>
  dplyr::summarise(
    n_cells = dplyr::n(),
    positive_cell_prevalence = mean(observed),
    PR_AUC = calculate_pr_auc(observed, predicted_probability),
    Brier_score = mean((observed - predicted_probability)^2),
    Brier_prevalence_baseline = mean((observed - mean(observed))^2),
    Brier_skill_vs_prevalence =
      1 - Brier_score / Brier_prevalence_baseline,
    .groups = "drop"
  )

cat("\nOverall spatial out-of-fold metrics:\n")
```

```
##
## Overall spatial out-of-fold metrics:
```

```
print(overall_metrics, n = Inf)
```

```
## # A tibble: 3 x 7
##   stage n_cells positive_cell_preval~1 PR_AUC Brier_score Brier_prevalence_bas~2
##   <chr>   <int>                  <dbl>  <dbl>       <dbl>                  <dbl>
## 1 Stag~   73236                0.00904 0.0177     0.00896                0.00896
## 2 Stag~   73236                0.0100  0.0274     0.0105                 0.00992
## 3 Stag~   73236                0.00904 0.0174     0.00897                0.00896
## # i abbreviated names: 1: positive_cell_prevalence,
## #   2: Brier_prevalence_baseline
## # i 1 more variable: Brier_skill_vs_prevalence <dbl>
```

```
# Moran's I of out-of-fold residuals.
# Eight nearest neighbouring cells define spatial neighbours.
morans_results <- list()

for (stage_name in unique(oof_predictions$stage)) {

  this_stage <- oof_predictions |>
    dplyr::filter(stage == stage_name)

  residuals_oof <- this_stage$observed - this_stage$predicted_probability
  coordinates_oof <- as.matrix(this_stage[, c("x", "y")])

  neighbours <- knn2nb(knearneigh(coordinates_oof, k = 8))
  spatial_weights <- nb2listw(neighbours, style = "W", zero.policy = TRUE)

  morans_results[[stage_name]] <- moran.test(
    residuals_oof,
    spatial_weights,
    zero.policy = TRUE
  )

  cat("\nOut-of-fold residual Moran's I for", stage_name, ":\n")
  print(morans_results[[stage_name]])
}
```

```
##
## Out-of-fold residual Moran's I for Stage_I :
##
##  Moran I test under randomisation
##
## data:  residuals_oof
## weights: spatial_weights
##
## Moran I statistic standard deviate = 7.4279, p-value = 5.518e-14
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      1.363280e-02     -1.365467e-05      3.375297e-06
```

```
##
##
## Out-of-fold residual Moran's I for Stage_II :
##
##  Moran I test under randomisation
##
## data:  residuals_oof
## weights: spatial_weights
##
## Moran I statistic standard deviate = 36.689, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      6.739946e-02     -1.365467e-05      3.376025e-06
##
##
## Out-of-fold residual Moran's I for Stage_III :
##
##  Moran I test under randomisation
##
## data:  residuals_oof
## weights: spatial_weights
##
## Moran I statistic standard deviate = 10.274, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      1.886264e-02     -1.365467e-05      3.375307e-06
```

6. Refit final models using all Gabon cells, map final probabilities, and print coefficients

```
final_models <- list()
final_probability_maps <- list()
coefficient_tables <- list()

for (outcome_name in outcome_names) {

  cat("\nFitting final all-Gabon model for", outcome_name, "\n")

  final_formula <- update(
    gam_formula,
    paste(outcome_name, "~ .")
  )

  final_model <- bam(
    formula = final_formula,
    family = binomial(link = "logit"),
    data = model_df,
    method = "fREML",
    discrete = TRUE
  )

  final_models[[outcome_name]] <- final_model
```

```r
  # Final predicted probability for every valid covariate-grid cell.
  model_df$final_probability <- as.numeric(
    predict(final_model, newdata = model_df, type = "response")
  )

  probability_raster <- template_raster
  values(probability_raster) <- NA_real_

  cell_numbers <- cellFromXY(
    probability_raster,
    model_df[, c("x", "y")]
  )

  values(probability_raster)[cell_numbers] <- model_df$final_probability
  names(probability_raster) <- paste0(outcome_name, "_occurrence_probability")

  final_probability_maps[[outcome_name]] <- probability_raster

  # Parametric covariate coefficients and odds ratios.
  coefficient_table <- as.data.frame(summary(final_model)$p.table)
  coefficient_table$term <- rownames(coefficient_table)
  rownames(coefficient_table) <- NULL

  coefficient_table <- coefficient_table |>
    dplyr::filter(term != "(Intercept)") |>
    dplyr::mutate(
      odds_ratio = exp(Estimate),
      stage = outcome_name
    ) |>
    dplyr::select(stage, term, Estimate, `Std. Error`, `z value`,
                  `Pr(>|z|)`, odds_ratio)

  coefficient_tables[[outcome_name]] <- coefficient_table

  cat("\nStandardized covariate coefficients for", outcome_name, ":\n")
  print(coefficient_table, row.names = FALSE)

  cat("\nFull GAM summary, including the spatial smooth:\n")
  print(summary(final_model))
}
```

```
##
## Fitting final all-Gabon model for Stage_I
##
## Standardized covariate coefficients for Stage_I :
##    stage                                    term    Estimate Std. Error
##  Stage_I        z_B_distance_to_Forest_Concessions -0.19670851 0.08310314
##  Stage_I z_B_distance_to_InternationalGabonBorders  0.34126583 0.10825447
##  Stage_I          z_B_distance_to_MajorGabonCities -1.03645218 0.07548320
##  Stage_I               z_B_distance_to_majorRoads  0.22598475 0.08585062
##  Stage_I            z_B_distance_to_ProtectedAreas -0.22607192 0.06775923
##  Stage_I                 z_B_distance_to_waterways -0.07486883 0.05633589
##  Stage_I                             z_B_LAI_2024  0.07118003 0.05447095
```

```
##  Stage_I                      z_B_Population_2020  0.02255359 0.01833112
##  Stage_I                           z_B_SRTM_2000  0.08729670 0.09206441
##      z value     Pr(>|z|) odds_ratio
##   -2.3670407 1.793096e-02  0.8214300
##    3.1524410 1.619115e-03  1.4067271
##  -13.7308983 6.630630e-43  0.3547109
##    2.6323019 8.480846e-03  1.2535565
##   -3.3364004 8.487082e-04  0.7976607
##   -1.3289723 1.838571e-01  0.9278652
##    1.3067521 1.912969e-01  1.0737745
##    1.2303443 2.185682e-01  1.0228098
##    0.9482134 3.430208e-01  1.0912204
##
## Full GAM summary, including the spatial smooth:
##
## Family: binomial
## Link function: logit
##
## Formula:
## Stage_I ~ z_B_distance_to_Forest_Concessions + z_B_distance_to_InternationalGabonBorders +
##     z_B_distance_to_MajorGabonCities + z_B_distance_to_majorRoads +
##     z_B_distance_to_ProtectedAreas + z_B_distance_to_waterways +
##     z_B_LAI_2024 + z_B_Population_2020 + z_B_SRTM_2000 + s(x,
##     y, k = 20, bs = "tp")
##
## Parametric coefficients:
##                                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)                           -5.07812    0.05453 -93.128  < 2e-16
## z_B_distance_to_Forest_Concessions    -0.19671    0.08310  -2.367 0.017931
## z_B_distance_to_InternationalGabonBorders  0.34127    0.10825   3.152 0.001619
## z_B_distance_to_MajorGabonCities      -1.03645    0.07548 -13.731  < 2e-16
## z_B_distance_to_majorRoads             0.22598    0.08585   2.632 0.008481
## z_B_distance_to_ProtectedAreas        -0.22607    0.06776  -3.336 0.000849
## z_B_distance_to_waterways             -0.07487    0.05634  -1.329 0.183857
## z_B_LAI_2024                           0.07118    0.05447   1.307 0.191297
## z_B_Population_2020                    0.02255    0.01833   1.230 0.218568
## z_B_SRTM_2000                          0.08730    0.09206   0.948 0.343021
##
## (Intercept)                           ***
## z_B_distance_to_Forest_Concessions    *
## z_B_distance_to_InternationalGabonBorders **
## z_B_distance_to_MajorGabonCities      ***
## z_B_distance_to_majorRoads            **
## z_B_distance_to_ProtectedAreas        ***
## z_B_distance_to_waterways
## z_B_LAI_2024
## z_B_Population_2020
## z_B_SRTM_2000
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df Chi.sq  p-value
## s(x,y) 13.39  16.36  42.56 0.000696 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.00683   Deviance explained = 6.07%
## fREML = 3600.4  Scale est. = 1         n = 73236
##
## Fitting final all-Gabon model for Stage_II
##
## Standardized covariate coefficients for Stage_II :
##     stage                                     term    Estimate Std. Error
##  Stage_II        z_B_distance_to_Forest_Concessions -0.15570659 0.07457811
##  Stage_II z_B_distance_to_InternationalGabonBorders  0.06525187 0.09399352
##  Stage_II          z_B_distance_to_MajorGabonCities -1.06713984 0.06961835
##  Stage_II               z_B_distance_to_majorRoads  0.28890162 0.07934347
##  Stage_II            z_B_distance_to_ProtectedAreas -0.23559749 0.06078353
##  Stage_II                 z_B_distance_to_waterways -0.25026539 0.05633908
##  Stage_II                             z_B_LAI_2024  0.11928145 0.04977763
##  Stage_II                        z_B_Population_2020  0.01596981 0.01880809
##  Stage_II                               z_B_SRTM_2000  0.15229934 0.08550463
##      z value     Pr(>|z|) odds_ratio
##   -2.0878324 3.681295e-02  0.8558103
##    0.6942166 4.875464e-01  1.0674278
##  -15.3284273 4.937979e-53  0.3439910
##    3.6411518 2.714210e-04  1.3349604
##   -3.8760090 1.061838e-04  0.7900986
##   -4.4421279 8.907360e-06  0.7785941
##    2.3962862 1.656215e-02  1.1266870
##    0.8490929 3.958296e-01  1.0160980
##    1.7811824 7.488266e-02  1.1645088
##
## Full GAM summary, including the spatial smooth:
##
## Family: binomial
## Link function: logit
##
## Formula:
## Stage_II ~ z_B_distance_to_Forest_Concessions + z_B_distance_to_InternationalGabonBorders +
##     z_B_distance_to_MajorGabonCities + z_B_distance_to_majorRoads +
##     z_B_distance_to_ProtectedAreas + z_B_distance_to_waterways +
##     z_B_LAI_2024 + z_B_Population_2020 + z_B_SRTM_2000 + s(x,
##     y, k = 20, bs = "tp")
##
## Parametric coefficients:
##                                         Estimate Std. Error z value Pr(>|z|)
## (Intercept)                             -4.97765    0.05120 -97.228  < 2e-16
## z_B_distance_to_Forest_Concessions      -0.15571    0.07458  -2.088 0.036813
## z_B_distance_to_InternationalGabonBorders  0.06525    0.09399   0.694 0.487546
## z_B_distance_to_MajorGabonCities        -1.06714    0.06962 -15.328  < 2e-16
## z_B_distance_to_majorRoads               0.28890    0.07934   3.641 0.000271
## z_B_distance_to_ProtectedAreas          -0.23560    0.06078  -3.876 0.000106
## z_B_distance_to_waterways               -0.25027    0.05634  -4.442 8.91e-06
## z_B_LAI_2024                             0.11928    0.04978   2.396 0.016562
## z_B_Population_2020                      0.01597    0.01881   0.849 0.395830
## z_B_SRTM_2000                            0.15230    0.08550   1.781 0.074883
```

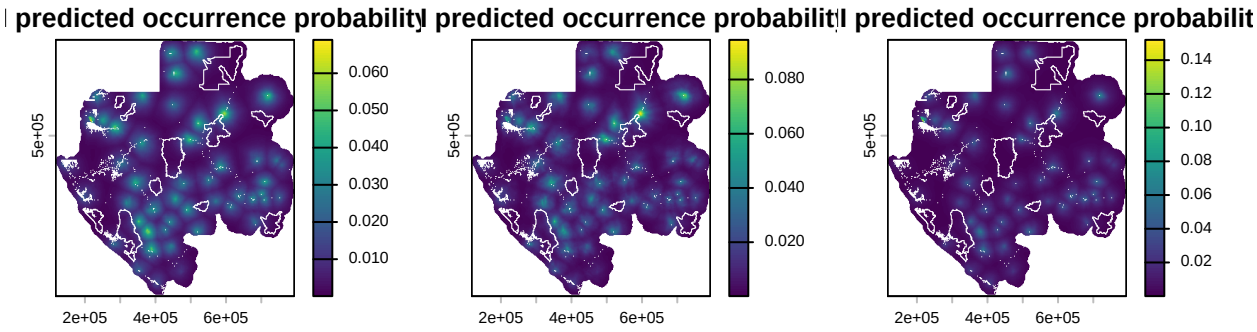
```
##
## (Intercept)                                ***
## z_B_distance_to_Forest_Concessions         *
## z_B_distance_to_InternationalGabonBorders
## z_B_distance_to_MajorGabonCities           ***
## z_B_distance_to_majorRoads                 ***
## z_B_distance_to_ProtectedAreas             ***
## z_B_distance_to_waterways                  ***
## z_B_LAI_2024                               *
## z_B_Population_2020
## z_B_SRTM_2000                              .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##         edf Ref.df Chi.sq  p-value
## s(x,y) 11.7  14.85  43.01 0.000139 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.0102   Deviance explained = 6.35%
## fREML = 3899.5  Scale est. = 1         n = 73236
##
## Fitting final all-Gabon model for Stage_III
##
## Standardized covariate coefficients for Stage_III :
##      stage                                      term     Estimate Std. Error
##  Stage_III        z_B_distance_to_Forest_Concessions -0.284988973 0.08772861
##  Stage_III z_B_distance_to_InternationalGabonBorders  0.374213666 0.11654033
##  Stage_III          z_B_distance_to_MajorGabonCities -1.152058883 0.07711137
##  Stage_III               z_B_distance_to_majorRoads  0.284038288 0.08729843
##  Stage_III            z_B_distance_to_ProtectedAreas -0.149437871 0.07162798
##  Stage_III                 z_B_distance_to_waterways -0.132050437 0.05897892
##  Stage_III                             z_B_LAI_2024 -0.009456281 0.05651237
##  Stage_III                        z_B_Population_2020  0.039134887 0.01453980
##  Stage_III                             z_B_SRTM_2000 -0.009366198 0.09557266
##       z value     Pr(>|z|) odds_ratio
##   -3.24852940 1.160032e-03  0.7520225
##    3.21102296 1.322634e-03  1.4538478
##  -14.94019529 1.804456e-50  0.3159855
##    3.25364707 1.139337e-03  1.3284838
##   -2.08630569 3.695094e-02  0.8611919
##   -2.23894298 2.515963e-02  0.8762968
##   -0.16733118 8.671095e-01  0.9905883
##    2.69157075 7.111641e-03  1.0399107
##   -0.09800081 9.219316e-01  0.9906775
##
## Full GAM summary, including the spatial smooth:
##
## Family: binomial
## Link function: logit
##
## Formula:
## Stage_III ~ z_B_distance_to_Forest_Concessions + z_B_distance_to_InternationalGabonBorders +
```

```
##     z_B_distance_to_MajorGabonCities + z_B_distance_to_majorRoads +
##     z_B_distance_to_ProtectedAreas + z_B_distance_to_waterways +
##     z_B_LAI_2024 + z_B_Population_2020 + z_B_SRTM_2000 + s(x,
##     y, k = 20, bs = "tp")
##
## Parametric coefficients:
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                             -5.177485   0.058659 -88.264  < 2e-16
## z_B_distance_to_Forest_Concessions      -0.284989   0.087729  -3.249  0.00116
## z_B_distance_to_InternationalGabonBorders  0.374214   0.116540   3.211  0.00132
## z_B_distance_to_MajorGabonCities        -1.152059   0.077111 -14.940  < 2e-16
## z_B_distance_to_majorRoads               0.284038   0.087298   3.254  0.00114
## z_B_distance_to_ProtectedAreas          -0.149438   0.071628  -2.086  0.03695
## z_B_distance_to_waterways               -0.132050   0.058979  -2.239  0.02516
## z_B_LAI_2024                            -0.009456   0.056512  -0.167  0.86711
## z_B_Population_2020                      0.039135   0.014540   2.692  0.00711
## z_B_SRTM_2000                           -0.009366   0.095573  -0.098  0.92193
##
## (Intercept)                             ***
## z_B_distance_to_Forest_Concessions      **
## z_B_distance_to_InternationalGabonBorders **
## z_B_distance_to_MajorGabonCities        ***
## z_B_distance_to_majorRoads              **
## z_B_distance_to_ProtectedAreas          *
## z_B_distance_to_waterways               *
## z_B_LAI_2024
## z_B_Population_2020                     **
## z_B_SRTM_2000
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df Chi.sq  p-value
## s(x,y) 14.53  17.22  56.93 5.59e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.00712   Deviance explained = 7.35%
## fREML = 3555.3  Scale est. = 1         n = 73236
```

```r
coefficient_tables <- dplyr::bind_rows(coefficient_tables)

# Map final occurrence probabilities.
par(mfrow = c(1, 3), mar = c(2, 2, 3, 4))

for (stage_name in names(final_probability_maps)) {
  plot(
    final_probability_maps[[stage_name]],
    main = paste(gsub("_", " ", stage_name), "predicted occurrence probability")
  )
}
```




```
par(mfrow = c(1, 1))
# Folder for exported maps
OUTPUT_DIR <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_
if (!dir.exists(OUTPUT_DIR)) {
  dir.create(OUTPUT_DIR, recursive = TRUE)
}
```

```
library(dplyr)
library(ggplot2)

# Prepare coefficients and approximate 95% confidence intervals
plot_data <- coefficient_tables |>
  mutate(
    covariate = case_when(
      term == "z_B_distance_to_Forest_Concessions" ~
        "Distance to forest concessions",
      term == "z_B_distance_to_InternationalGabonBorders" ~
        "Distance to international borders",
      term == "z_B_distance_to_MajorGabonCities" ~
        "Distance to major cities",
      term == "z_B_distance_to_majorRoads" ~
        "Distance to major roads",
      term == "z_B_distance_to_ProtectedAreas" ~
        "Distance to protected areas",
      term == "z_B_distance_to_waterways" ~
```

```
        "Distance to waterways",
      term == "z_B_LAI_2024" ~
        "Leaf area index",
      term == "z_B_Population_2020" ~
        "Population density",
      term == "z_B_SRTM_2000" ~
        "Elevation",
      TRUE ~ term
    ),

    stage = factor(
      stage,
      levels = c("Stage_I", "Stage_II", "Stage_III"),
      labels = c("Stage I", "Stage II", "Stage III")
    ),

    # Approximate 95% Wald confidence intervals
    lower_ci = Estimate - 1.96 * `Std. Error`,
    upper_ci = Estimate + 1.96 * `Std. Error`
  )

# Order covariates by their largest |coefficient| across the three stages
covariate_order <- plot_data |>
  group_by(covariate) |>
  summarise(
    max_abs_coefficient = max(abs(Estimate), na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(max_abs_coefficient) |>
  pull(covariate)

plot_data <- plot_data |>
  mutate(
    covariate = factor(covariate, levels = covariate_order)
  )

# Colours for the three crime stages
stage_colours <- c(
  "Stage I"   = "#1b9e77",
  "Stage II"  = "#d95f02",
  "Stage III" = "#7570b3"
)

# Slightly separate the three estimates within each covariate row
stage_dodge <- position_dodge(width = 0.6, reverse = TRUE)

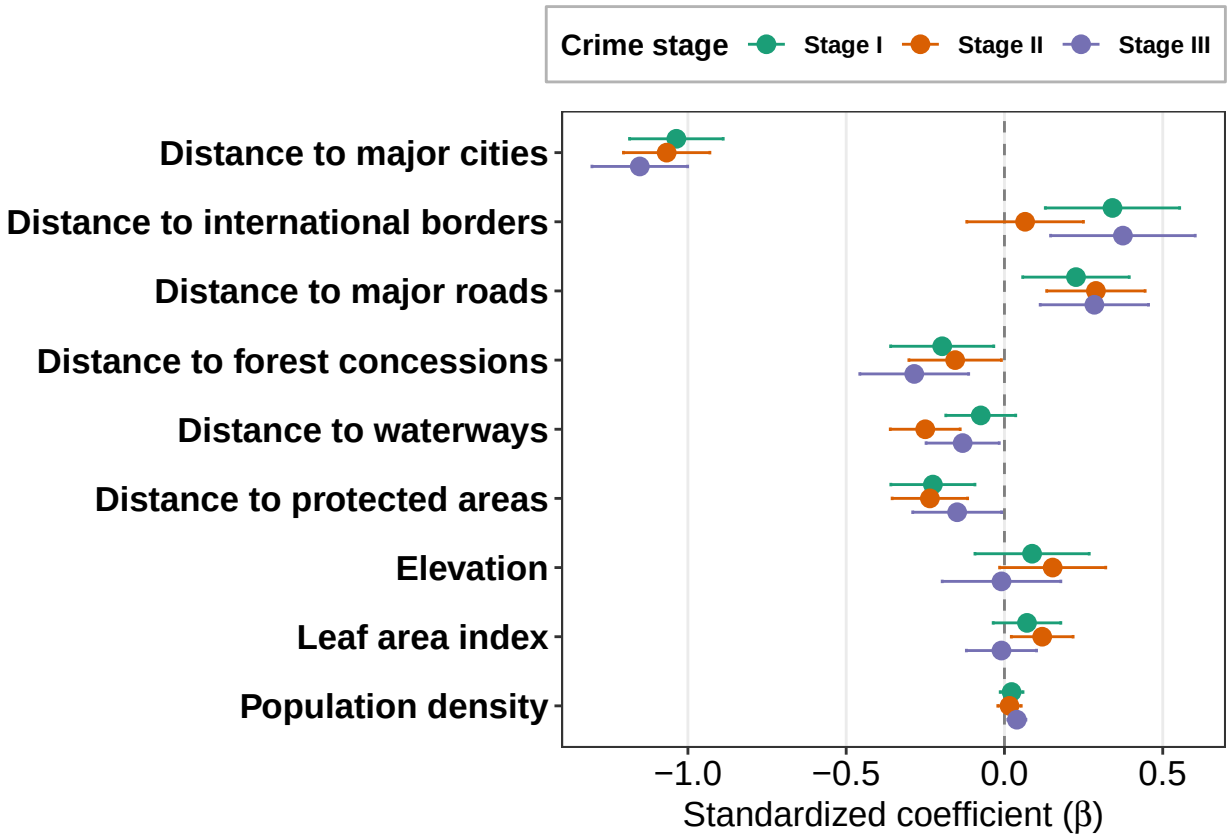
coefficient_plot <- ggplot(
  plot_data,
  aes(x = Estimate, y = covariate, colour = stage)
) +
  geom_vline(
    xintercept = 0,
    linetype = "dashed",
```

```
    colour = "grey50"
  ) +
  geom_errorbar(
    aes(xmin = lower_ci, xmax = upper_ci),
    orientation = "y",
    position = stage_dodge,
    width = 0.15,
    linewidth = 0.5
  ) +
  geom_point(
    position = stage_dodge,
    size = 2.8
  ) +
  scale_colour_manual(
    values = stage_colours,
    name = "Crime stage"
  ) +
  labs(
    x = expression("Standardized coefficient (" * beta * ")"),
    y = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(
  axis.text.x = element_text(size = 13, colour = "black"),
  axis.text.y = element_text(size = 13, face = "bold", colour = "black"),
  axis.title.x = element_text(size = 13, face = "bold", colour = "black"),

  legend.position = "top",
  legend.justification = "right",
  legend.direction = "horizontal",
  legend.background = element_rect(fill = "white", colour = "grey70"),
  legend.title = element_text(face = "bold", colour = "black"),
  legend.text = element_text(face = "bold", colour = "black"),

  panel.grid.minor = element_blank(),
  panel.grid.major.y = element_blank()
  )

# Display the plot
print(coefficient_plot)
```




```r
# Folder for the exported figure
OUTPUT_DIR <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_

if (!dir.exists(OUTPUT_DIR)) {
  dir.create(OUTPUT_DIR, recursive = TRUE)
}

figure_file <- file.path(
  OUTPUT_DIR,
  "GAM_standardized_coefficients_colour_by_stage.png"
)

if (file.exists(figure_file)) {
  suffix <- 1L
  repeat {
    candidate_file <- file.path(
      OUTPUT_DIR,
      paste0("GAM_standardized_coefficients_colour_by_stage_", suffix, ".png")
    )
    if (!file.exists(candidate_file)) break
    suffix <- suffix + 1L
  }
  figure_file <- candidate_file
}

ggsave(
```

```
  filename = figure_file,
  plot = coefficient_plot,
  width = 11,
  height = 7,
  units = "in",
  dpi = 300,
  bg = "white"
)

cat("Figure saved to:", figure_file, "\n")
```

```
## Figure saved to: C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pango
```